PicoBoot-WiFi: implementation plan

Status	Draft v1.1 (after phase 0), waiting for the owner's approval
Design	architecture.md, draft v1.2
Owner	Remko Kleinjan

Status

Version	Date	What changed and why
v0.1 draft	2026-11-08	First draft. It splits the design's phases (D28) into steps, assigns models and lists what's needed from the owner. The owner's equipment (a current meter, no oscilloscope or logic analyzer) and console (DOL-001 only) don't fit the design's scope-based measurements. Design points DP1–DP5 propose the changes.
v1.0	2026-11-08	The owner approved the draft with DP1–DP5 (”approved, go ahead with phase 0”). DP1–DP5 are now design decisions D31–D35 (design v1.1). Phase 0 starts.
v1.1	2026-11-08	Phase 0 as built (see its ”As built” note), and what its results change. **Phase 1:** a new step 1.3 for the glitch protection (D36), the last flash sector kept free (D37), the RP2350 stepping in the report, and a glitch check among the console tests. **Phase 2:** notes on lwIP (DHCP, send buffer) and the Tx power default. **Phase 5:** the 5 V recommendation (D38) in the README. Phase 1's steps are renumbered 1.1–1.12.

How this plan works

- **Steps:** every phase is split into steps. Each step has a ”done when” check.
- **Gates:** a phase starts only after the previous one is merged. Phase 0 can still change the design. If it does, the design gets a new version (v1.1, …) before phase 1 starts.
- **Branches:** one per phase: `phase-0-checks`, `phase-1-core`, `phase-2-wifi`, `phase-3-firmware-ota`, `phase-4-payload-ota`, `phase-5-release`. `main` holds only reviewed work.
- **Commits:** once this plan is approved, each step is committed on its phase branch without asking each time.
- **The end of a phase:**
  1. the owner's hardware tests;
  2. a code review on the branch (fixes go on the same branch);
  3. the design and this plan updated to match what was built;
  4. the owner's review;
  5. a merge into `main` with a merge commit, and the status lines saying ”merged”;
  6. the PDF exports regenerated (`docs/export-pdf.sh`).

Who does what

Who	Does
Opus 5.5 (this session)	The plan, the core in C (loader, records, injection, measurement, Wi-Fi blob hook, flash writer, OTA and trial logic, payload slots, httpd integration), CMake and CI, local builds, and reviews of all subagent work
Sonnet 5.5 subagents, each in its own git worktree	Independent parts with a written contract from Opus: the web UI, the JavaScript `.dol` converter, the Python tools (update-file builder, CLI client)
Fresh Opus 5.5 reviewer	The end-of-phase code review of the code this session wrote itself, so the review isn't anchored on the author's reading
Fable 5.1	The security review in phase 5. It wrote none of the code

Who	Does
Owner	Hardware tests on the DOL-001, pushes to GitHub (CI runs on push), reviews and approvals, merges

What's needed from the owner

What	When
A Pico 2 W for the console, ideally a second one for bench tests	Phase 0
The DOL-001 with the SD2SP2 and the HDMI adapter fitted, wired as today (3V3 feed)	Phase 0
A current meter or shunt in the Pico's 3.3 V feed	Phase 0
A phone (and ideally a second one with the other OS, for the captive-portal check) and a laptop	Phase 0, 2
A GitHub repository `r3mko/PicoBoot-WiFi` with Actions enabled; pushing each phase branch	Phase 1
Payload files to test with: Swiss r2119 and gekkoboot r9.4 (UF2 and `.dol`)	Phase 0, 4

No servers, DNS, accounts or tokens are needed.

Design points for approval

The owner's answers on 2026-11-08 (equipment, console, readout) don't fit three design decisions. The owner approved these points with the plan, and they are now design decisions D31–D35 (design v1.1).

DP1 → D31: the Pico measures its own boot timing (changes D10).

- **How:** a third, passive PIO state machine watches CS. It doesn't touch the two injection programs.
  - It sets a PIO IRQ flag on the first CS falling edge, and the CPU, which is idle during the injection, timestamps that flag.
  - It counts every CS pulse until the injection ends.
  - The DMA transfer count gives the number of 1 KB transfers delivered.
- **The margin:** time of the first CS minus time of ”PIO ready”.
  - A positive margin together with a successful boot proves the PIO was in time.
  - A short CS count shows missed pulses.
- **Why:** no oscilloscope or logic analyzer is available.
- **Reuse:** the same counters become the boot diagnostics of D22.
- **What it can't do:** it can't see the supply ramp before the RP2350 leaves reset. That doesn't matter, because the margin is measured from the Pico's own start.

DP2 → D32: a minimal Wi-Fi readout comes early (changes D28).

- **Phase 0:** the measurement build is PicoBoot-DX v0.10 plus the DP1 counters. After the injection it starts an access point with a plain-text report page. It also serves two load-test URLs, a large download (Tx load) and an upload sink (Rx load).
- **Phase 1:** brings the Wi-Fi blob hook (D14) and the same minimal page forward from phase 2. The blob doesn't fit inside a 256K slot, and the results have to come out of a closed console.

DP3 → D33: the rail check without a scope (changes D28 phase 0).

- **Average current:** measured with the meter in four states:
  - AP off;
  - AP idle;
  - a client connected;
  - the download running.
- **Rail sag:** a functional stress test replaces the scope. The large download runs at full Tx power while Swiss loads a game from the SD2SP2, with the HDMI adapter on. What to watch for:
  - console glitches;
  - video artefacts;
  - a Pico brown-out (`HAD_B0R`, counted in the report);

  - Wi-Fi drops.

DP4 → D34: the DOL-001 is the test console.

- All hardware tests run on the owner's DOL-001 with the SD2SP2 and the HDMI adapter.
- The README says the DOL-101 is expected to work (same IPL path as PicoBoot) but is untested.

DP5 → D35: no pre-releases; the first GitHub release is 1.0.0 (changes D24).

- **Test builds:** come from CI artifacts on the pushed phase branch, or from Opus's local Docker build, which uses the same SDK and toolchain as CI.
- **Versions:** builds carry 0.N.0-dev (N = the phase) plus the git hash.

Repository layout

```
CMakeLists.txt              loader + firmware targets, install UF2, update file
pico_sdk_2_import.cmake
src/common/                 layout.h (the flash map, D6), records, boot info, version
src/loader/                 the loader (D7)
src/firmware/               injection, measurement, Wi-Fi, web, OTA
src/firmware/pio/           PIO programs (injection from PicoBoot-DX, CS counter)
linker/                     linker overrides (loader 16K, firmware 256K slot)
web/                        static UI sources (gzipped into the firmware)
tools/                      Python: process_ipl, uf2 packer, update builder, CLI
tests/                      Python tests, PIO simulator, host C tests, layout check
hwtest/dx-measure/          phase 0 measurement build (PicoBoot-DX + DP1 + DP2)
docker/Dockerfile         local build image: SDK 2.3.1, picotool 2.3.1, arm-none-eabi-gcc
docs/                       design, plan, research, API, update-file format
```

`src/common/layout.h` is the only place the flash map is written down. The linker scripts, the tools and the layout check read it.

Phase 0: hardware checks

Branch: `phase-0-checks`. **Goal:** measure what the design couldn't know, before any core code exists.

#	Step	Who	Done when
0.1	**Measurement build** `hwtest/dx-measure`. Start from PicoBoot-DX v0.10 (pico2_w) and add: the DP1 counter state machine; after the injection, the pico-examples access point (WPA2, test password) with `/report` (text), `/stream` (large download) and `/sink` (upload). The report shows the margin, the CS count, transfers delivered, injection complete or stalled, reset reason (HAD_POR/HAD_BOR), time to Wi-Fi up and uptime.	Opus	Builds in Docker; the PIO simulator shows the injection unchanged with the counter running
0.2	**Bench check**: Pico 2 W on USB, no console	Owner	The AP is visible; `/report` says ”no CS seen”; `/stream` downloads
0.3	**Cold boots in the console**, 3V3 wiring as today: 10× gekkoboot r9.4, 10× Swiss r2119, reading `/report` after each	Owner	20 results recorded; the radio works on the console's 3.3 V; every boot good
0.4	**Warm board**: 5 boots after 30 minutes of play	Owner	Recorded, compared with 0.3
0.5	**Current**: AP off (during the injection), AP idle, client connected, `/stream` running (DP3)	Owner	Four values recorded
0.6	**Range**: phone RSSI at 1, 3 and 5 m with the console closed, in two board positions	Owner	Recorded

#	Step	Who	Done when
0.7	**Stress test** (DP3): `/stream` while Swiss loads a game from the SD2SP2, HDMI adapter on	Owner	Glitches, video, brown-outs and Wi-Fi drops recorded
0.8	**Results** in `docs/research/phase0-results.md`, design changes (if any) as a new design version	Opus	The owner approves the results and any design change

As built (2026-11-08):

- **Moved forward from phase 1** (needed for the measurement build): `docker/Dockerfile`, `tools/process_ipl.py`, gekkoboot r9.4 in `assets/ipl/`, and `tests/pio_sim.py`.
- **Added to the measurement build** while testing:
  - D8's warm-reboot rule, with *Reboot* buttons to test it;
  - test switches: built-in IPL, Wi-Fi 60 s late, CS pull-up;
  - a check of the last flash page for the E10 block;
  - a second, clock-qualified CS counter;
  - the pin levels at "PIO ready".
- **Step 0.3b added:** the glitch series after boot 12 of step 0.3 (parts 1–3).
- **Step 0.5:** measured the 3V3 voltage instead of the current, because the owner's setup can't put a meter in series.
- **Bug found on the bench and fixed:** lwIP drops DHCP requests when `LWIP_DHCP` is off, even for a DHCP *server*.

Code review of phase 0 (a fresh Opus 5.5 reviewer, 2026-11-08):

- **Overall:** no serious bugs. The injection programs are byte-identical to PicoBoot-DX v0.10, the build is clean under `-Wall -Wextra`, the tests pass, and the warm-reboot and flash-write constraints hold.
- **The measurement build stays as it was measured.** Its findings are requirements for the phase 1 code that reuses it:

#	Finding	Where it's handled
1	The clock-qualified count must check that CS is still low when CLK rises (`jmp pin`); otherwise a glitch plus a CLK pulse while CS is high still counts. Already corrected in the PicoBoot-DX handoff	1.3, 1.4: in the injection and the counters, with simulator cases for CLK while CS is high (`pio_sim` gets `JMP !X` and `JMP PIN`)
2	Recovery from a torn record write erased the newer records in the same sector	1.5: erase only at a sector start; host test
3	The phase 0 power-on rule also injects on BOR and RUN; D8 is HAD_POR only. Keep POWMAN `scratch[0]` for the marker	1.6
4	A failed `tcp_write` with nothing in flight is never retried	2.2: retry from the poll callback (or lwIP httpd handles it)
5	A client FIN cuts the response short	2.2: finish the response, then close
6	The `tcp_abort` fallback after `tcp_close` would be unsafe inside a callback	2.2: no such fallback, or return `ERR_ABRT`
7	Raw and clocked counts are read at different moments, so a false GLITCH is possible	1.4: snapshot while CS is high, or re-read until stable
8	`dma_channel_abort()` without first clearing the channel's EN bit (RP2350-E5)	1.7
9	The counters use PIO IRQ flags 0 and 1, which are only safe outside the injection's PIO block	1.4: use flags the injection doesn't use, or keep them in another block
—	Test gaps: the counter's X set-up and read path, and coupling between the counters and the injection, aren't exercised	1.4

Phase 1: core

Branch: `phase-1-core`. **Done when (D28):**

- the console boots a payload through the loader on a cold and on a warm board;
- a warm reboot doesn't inject;
- the boot-time budget is met (D10);
- the clock is set (D27).

#	Step	Who	Done when
1.1	**Skeleton**: CMake with the loader and firmware targets, `layout.h` (last flash sector kept free, D37), linker overrides (loader ≤16K, firmware ≤256K), CI (ubuntu-26.04: build, tests, layout check) with the phase 0 `docker/Dockerfile` setup, LICENSE (GPL-2.0-only, D29), README stub	Opus	CI builds both ELFs; the layout check passes
1.2	**Injection port** from PicoBoot-DX: PIO programs, `pio.c`, payload check (base address as a parameter), `ipl.h` from `tools/process_ipl.py` (both from phase 0), and the remaining PicoBoot-DX tests next to `tests/pio_sim.py`	Opus	CI tests green, PIO simulator included
1.3	**Glitch protection** (D36): • CS pull-up before the PIO starts; • a clock-qualified count of the 224 skipped pulses in `on_transfer`, CLK rising while CS is still low (`jmp pin`), as in the PicoBoot-DX handoff (`docs/handoff/`). Simulator tests: with a CS glitch without clocks before the IPL read, PicoBoot-DX's programs shift the payload (phase 0's test) and the new ones don't; without a glitch, the output is bit-identical; every address clock of the first payload transfer is seen	Opus	Simulator tests green, including the phase 0 glitch case
1.4	**CS counters and boot report** (D31): the raw and clocked counters from phase 0 as firmware code, plus the RP2350 stepping (for RP2350-E9)	Opus	The simulator test counts the right pulses and transfers
1.5	**Records** for boot control and settings (D23): two sectors, sequence number, CRC32, newest valid record wins	Opus	Host C tests pass, including a torn write that falls back to the previous record
1.6	**Loader** (D7, D8): • power-on rule (`HAD_POR` + POWMAN scratch marker); • slot choice as one pure function; • boot info in POWMAN scratch; • `rom_chain_image()`; • the other slot if one is empty, BOOTSEL if both are; • no clock setup (`pico_minimize_runtime`)	Opus	Host tests cover the decision table; the size check is ≤16K
1.7	**Firmware start-up**: read boot info; power-on → inject payload 1, 2 or built-in (payload read through the untranslated XIP window); warm → no injection	Opus	On the bench, the loader chains into slot A and `/report` shows a warm boot

#	Step	Who	Done when
1.8	**Wi-Fi blob hook** (D14, DP2): the blob is loaded from Wi-Fi A/B at its fixed address. The minimal AP and `/report` page from 0.1 run inside the slot, with a *reboot* button for the warm-reboot test	Opus	The AP comes up from a slot build with the blob outside the image
1.9	**Install UF2**: `tools/mkuf2.py` packs absolute regions (rp2350-arm-s): loader, blank boot control and settings, Wi-Fi A (Wi-Fi-only blob, built from the SDK header) and slot A, without picotool's E10 block (D37). CI artifact `picoboot-wifi-install.uf2`	Opus	Python tests for the packer; the UF2 parser shows the right ranges and nothing at or above `0x3FF000`; a bench install over PicoBoot-DX keeps payload 1
1.10	**Console tests**: • 10 cold boots each with gekkoboot and Swiss; • warm board; • *reboot* from the page while a game runs (the console keeps running and the report says ”warm, no injection”); • the glitch check: 40 cold boots with a test switch that restores PicoBoot's CS pull-down, where every boot must start and any glitch must show as survived in the report. Compare the margin with phase 0	Owner	Results recorded; the margin difference gives the loader's cost; no blue screens
1.11	**Budget and clock**: set the D10 budget and the D27 clock and flash divider from phase 0 and 1.10. If the loader is too slow, the R1 fallback (loader starts the PIO)	Opus + owner	Values written into the design; budget met
1.12	**End of phase**: fresh Opus review, fixes, design and plan updated, merge	Opus, owner	Merged

Phase 2: Wi-Fi

Branch: `phase-2-wifi`. **Done when (D28):** a phone reaches the status page from a closed console.

#	Step	Who	Done when
2.1	**Settings** on the records from 1.4: SSID, password, channel, country, Tx power, auto-off, LED; defaults from D30	Opus	Host tests; settings survive a reboot
2.2	**Network**: AP with WPA2-AES (D30), DHCP, DNS answering every name with `192.168.4.1`, redirects for the phone probe URLs, lwIP httpd with custom files. From phase 0: keep `LWIP_DHCP` on (without it lwIP drops DHCP requests), and size the TCP send buffer for throughput (8 × MSS gave 0.4–0.95 MB/s)	Opus	iOS and Android open the page by themselves on the bench
2.3	**API contract** `docs/api.md` (status, settings, diagnostics) and the JSON handlers	Opus	Handlers answer as specified
2.4	**Web UI**: plain HTML/CSS/JS, status, diagnostics and settings pages, default-password banner (D19), gzip build step	Sonnet (worktree)	Works against the API on the bench; under about 30 KB gzipped; Opus reviewed

#	Step	Who	Done when
2.5	**Diagnostics** (D22): the boot report from 1.3, plus the loader decision, the payload choice and why, time to Wi-Fi up	Opus	Shown on the status page
2.6	**Auto-off** (D20), **Tx power** (the `qtxpower` iovar), **LED** patterns (D26). The default Tx power is set lower than the maximum, since phase 0 found plenty of range (−42 dBm at 5 m)	Opus	Auto-off after 10 minutes without a client; Tx power changes the RSSI
2.7	**Console tests**: captive portal on both phones, settings, banner, auto-off, Tx power against range and current; injection results the same as in phase 1	Owner	Recorded
2.8	**End of phase**: fresh Opus review, fixes, docs, merge	Opus, owner	Merged

Phase 3: firmware OTA

Branch: `phase-3-firmware-ota`. **Done when (D28):** an update, a failed trial and a switch all work in the console without USB.

#	Step	Who	Done when
3.1	**Update-file format** `docs/update-format.md` (D11): header, slot image, optional blob, SHA-256s	Opus	Format reviewed by the owner
3.2	**Update-file builder** `tools/mkupdate.py`, with tests and a CI artifact	Sonnet (worktree)	Tests green; the output matches the format; Opus reviewed
3.3	**Flash writer**: streaming POST with a manual TCP window, sector erase and program through `flash_safe_execute()`, SHA-256 on the fly (`pico_sha256`) and after reading back, Wi-Fi blob pairing (D14), boot control ”pending”	Opus	Bench: an update lands in slot B and verifies; a corrupted file is refused
3.4	**Trial, keep and fallback** (D12, D13): • kept after a good power-on injection plus Wi-Fi, or on *Keep*; • ”trial failed”; • crash-loop counter; • watchdog and HardFault → reboot; • loader decision tests extended	Opus	Host tests; on the bench a crash build falls back
3.5	**Firmware page**: upload with progress, both slots, *Keep*, *Boot slot X*	Sonnet (worktree)	Works on the bench; Opus reviewed
3.6	**CLI** `tools/picoboot-wifi.py status / update` (standard library only)	Sonnet (worktree)	Updates a bench board; Opus reviewed
3.7	**Test builds**: a firmware that crashes at start-up and one that crashes after 5 s, as CI artifacts	Opus	Built

#	Step	Who	Done when
3.8	**Console tests**: • update A→B, kept after a power-on; • crash build → fallback; • switching slots; • power cut during an upload → the old firmware boots; • update with a different blob (the Bluetooth variant as a stand-in)	Owner	Recorded
3.9	**End of phase**: fresh Opus review, fixes, docs, merge	Opus, owner	Merged

Phase 4: payload OTA

Branch: `phase-4-payload-ota`. **Done when (D28):** Swiss and gekkoboot uploaded and chosen over Wi-Fi.

#	Step	Who	Done when
4.1	**Payload upload** (D15, D16): streaming UF2 parser (any family, blocks from `0x10080000`), raw payload, checks, write to the slot that isn't booting, metadata (name, size, SHA-256, fingerprint) in boot control	Opus	Host tests for the parser with the real Swiss and gekkoboot UF2s; bench upload
4.2	**USB rule** (D17): fingerprint check before the injection, record after it	Opus	A USB install over a payload-2 choice boots payload 1
4.3	**"Original IPL once"**: a one-shot flag in boot control, cleared at the next power-on	Opus	Bench and host tests
4.4	**Backups**: streaming downloads of payload 1/2, the firmware slot and settings	Opus	Downloaded SHA-256 matches the status page
4.5	**`.dol` converter** in JavaScript (port of `process_ipl.py` and the UF2 payload format), with a Node test in CI against the Python output	Sonnet (worktree)	Byte-identical to Python for gekkoboot r9.4 and Swiss r2119; Opus reviewed
4.6	**Payload page**: upload `.dol`/`.uf2`/`.bin`, choose, delete, download	Sonnet (worktree)	Works on the bench; Opus reviewed
4.7	**CLI** payload commands: upload, choose, convert	Sonnet (worktree)	Works on the bench; Opus reviewed
4.8	**Console tests**: • Swiss and gekkoboot as UF2 and as `.dol`; • choosing each; • the USB rule; • "original IPL once"; • power cut during a payload upload; • backup	Owner	Recorded
4.9	**End of phase**: fresh Opus review, fixes, docs, merge	Opus, owner	Merged

Phase 5: release 1.0

Branch: `phase-5-release`. **Done when (D28):** the owner's hardware test passes on a fresh install and on an upgrade from PicoBoot-DX.

#	Step	Who	Done when
5.1	**Security review** of the whole network side: httpd handlers, upload parsers, flash-writer bounds, settings input, DNS and DHCP	Fable 5.1	Findings reported; fixes on the branch, reviewed by Opus
5.2	**README** (install, upgrade from PicoBoot-DX, wiring with the 5 V recommendation of D38, default password, recovery, DOL-101 note), `docs/api.md` final	Opus	Owner reviewed
5.3	**Release workflow**: on a tag, build and attach the install UF2, the update file and checksums	Opus	A test tag on a fork produces the assets
5.4	**Acceptance** on the DOL-001: fresh install, upgrade from PicoBoot-DX, OTA from a 0.4.0-dev build to 1.0.0	Owner	Passed
5.5	**Merge and tag 1.0.0**	Owner	Released

Local builds and CI

- **Local:** Opus builds in Docker (`docker/Dockerfile`: Ubuntu, SDK 2.3.1, picotool 2.3.1, arm-none-eabi-gcc), the same setup the size research used (sizes.md). Test UF2s go to the owner from there or from CI.
- **CI:** builds, then runs every test:
  - Python tools;
  - the PIO simulator;
  - host C tests;
  - the layout check;
  - the Node test for the JS converter (from phase 4).

  The CI run is the authority on whether a step is done.